ASSIGNMENT – 11.5

2303A51091
Batch : 29

Task Description #1 – Stack Implementation
Task: Use AI to generate a Stack class with push, pop, peek, and is_empty methods.
Sample Input Code:
class Stack:
pass
Expected Output:
• A functional stack implementation with all required methods and docstrings.

task1.py > ...

```python
class Stack:
    """
    A simple Stack implementation using Python lists.
    Follows the Last In First Out (LIFO) principle.
    """

    def __init__(self):
        """
        Initializes an empty stack.
        """
        # Internal list to store stack elements
        self.items = []

    def push(self, item):
        """
        Adds an item to the top of the stack.

        Args:
            item: The element to be added to the stack.
        """
        # Append the item to the end of the list (top of stack)
        self.items.append(item)

    def pop(self):
        """
        Removes and returns the top item of the stack.

        Returns:
            The item at the top of the stack.

        Raises:
            IndexError: If the stack is empty.
        """
        # Remove and return the last item if stack is not empty
        if self.is_empty():
```

Output:

```
arepallyramcharan@Arepallys-MacBook-Air assignment-11.5.py % /usr/bin/python3 "/Users/arepallyr
amcharan/ai assistent.py/assignment-11.5.py/task1.py"
Top element: 30
Popped: 30
Popped: 20
Is stack empty? False
Popped: 10
Is stack empty? True
arepallyramcharan@Arepallys-MacBook-Air assignment-11.5.py %
```

Observation:

1. AI assistance helps in quickly designing the stack structure and operations.
2. The stack follows the **LIFO (Last In First Out)** principle.
3. Operations like push, pop, peek, and empty-check ensure proper stack functionality.
4. Documentation improves code readability and understanding.
5. AI-generated implementations help beginners understand data structure concepts efficiently.

Task Description #2 – Queue Implementation
Task: Use AI to implement a Queue using Python lists.
Sample Input Code:
class Queue:
pass
Expected Output:
• FIFO-based queue class with enqueue, dequeue, peek, and size methods.

task2.py > ...

```python
class Queue:
    """
    A Queue implementation using Python lists.
    Follows FIFO (First In First Out) principle.
    """

    def __init__(self):
        """
        Initializes an empty queue.
        """
        self.items = []  # List to store queue elements

    def enqueue(self, item):
        """
        Adds an item to the end of the queue.

        Args:
            item: The item to be added to the queue.
        """
        self.items.append(item)  # Append item to the end

    def dequeue(self):
        """
        Removes and returns the item from the front of the queue.

        Returns:
            The item at the front of the queue.

        Raises:
            IndexError: If the queue is empty.
        """
        if self.size() == 0:
            raise IndexError("dequeue from empty queue")
        return self.items.pop(0)  # Remove and return the first item
```

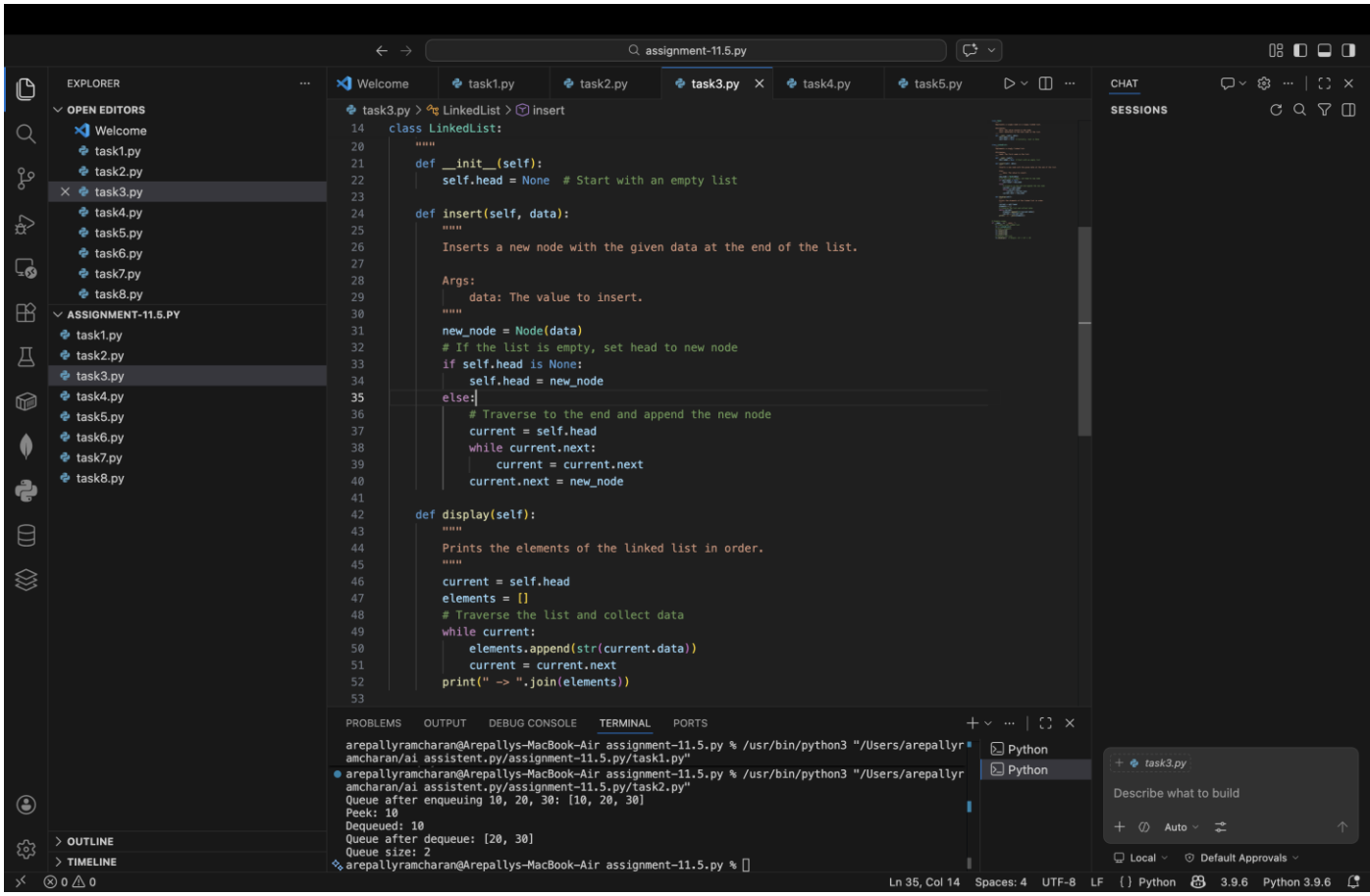
Output:

```
arepallyramcharan@Arepallys-MacBook-Air assignment-11.5.py % /usr/bin/python3 "/Users/arepallyr
amcharan/ai assistent.py/assignment-11.5.py/task2.py"
Queue after enqueuing 10, 20, 30: [10, 20, 30]
Peek: 10
Dequeued: 10
Queue after dequeue: [20, 30]
Queue size: 2
arepallyramcharan@Arepallys-MacBook-Air assignment-11.5.py %
```

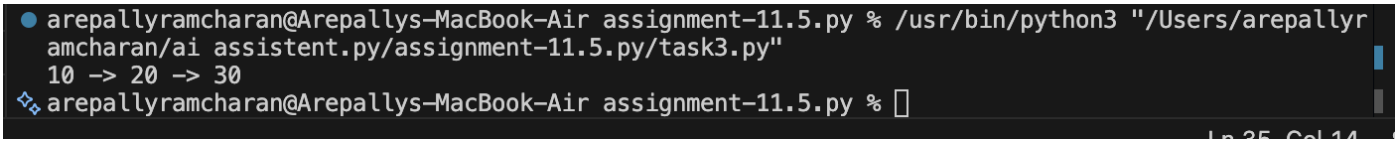
Observation:

- The queue data structure follows the **FIFO (First In First Out)** principle.
- AI guidance simplifies the implementation of enqueue and dequeue operations.
- Front element access helps identify the next element to be removed.
- Size calculation ensures proper monitoring of queue elements.
- AI-assisted development speeds up coding while maintaining correctness.

Task Description #3 – Linked List
Task: Use AI to generate a Singly Linked List with insert and display methods.
Sample Input Code:
class Node:
pass
class LinkedList:
pass
Expected Output:
• A working linked list implementation with clear method documentation.



Output:



Observation:

- A singly linked list consists of nodes connected using references.
- Each node contains data and a pointer to the next node.
- Insertion allows dynamic addition of elements in the list.
- Traversal helps display and process elements sequentially.
- AI assistance simplifies the design and implementation of linked lists.
-

Task Description #4 – Hash Table

Task: Use AI to implement a hash table with basic insert, search, and delete methods.

Sample Input Code:

class HashTable:

pass

Expected Output:

• Collision handling using chaining, with well-commented methods.

task4.py > ...

```python

class HashTable:
    """
    HashTable implementation using chaining for collision handling.
    """

    def __init__(self, size=10):
        """
        Initialize the hash table with a fixed number of buckets.
        :param size: Number of buckets in the hash table.
        """
        self.size = size
        self.buckets = [[] for _ in range(size)]

    def _hash(self, key):
        """
        Compute the hash index for a given key.
        Uses Python's built-in hash function and modulo operation.
        :param key: The key to hash.
        :return: Index of the bucket.
        """
        # The hash function maps the key to an integer, then modulo size gives the bucket index.
        return hash(key) % self.size

    def insert(self, key, value):
        """
        Insert a key-value pair into the hash table.
        If the key already exists, update its value.
        :param key: The key to insert.
        :param value: The value associated with the key.
        """
        index = self._hash(key)
        bucket = self.buckets[index]
        # Check if key exists and update value if found
        for i, (k, v) in enumerate(bucket):
```

Output:

```
arepallyramcharan@Arepallys-MacBook-Air assignment-11.5.py % /usr/bin/python3 "/Users/arepallyr
amcharan/ai assistent.py/assignment-11.5.py/task4.py"
Search 'banana': 20
Delete 'apple': True
Search 'apple': None
arepallyramcharan@Arepallys-MacBook-Air assignment-11.5.py %
```

Observation:

- A hash table stores data using key-value pairs.
- A hash function converts keys into index positions.
- Collision handling techniques such as chaining improve efficiency.
- Insert, search, and delete operations enable fast data retrieval.
- AI support helps implement hashing techniques accurately.

Task Description #5 – Graph Representation
Task: Use AI to implement a graph using an adjacency list.
Sample Input Code:
class Graph:
pass
Expected Output:
• Graph with methods to add vertices, add edges, and display connections.

task5.py > Graph > add_vertex

```python
class Graph:
    """
    A simple Graph implementation using an adjacency list.
    The adjacency list is stored as a dictionary where each key is a vertex,
    and the value is a list of adjacent vertices.
    """

    def __init__(self):
        """
        Initializes an empty adjacency list.
        """
        self.adjacency_list = {}

    def add_vertex(self, vertex):
        """
        Adds a new vertex to the graph.

        Args:
            vertex: The vertex to be added.
        """
        if vertex not in self.adjacency_list:
            self.adjacency_list[vertex] = []
        # If the vertex already exists, do nothing.

    def add_edge(self, v1, v2):
        """
        Adds an edge between two vertices in the graph.

        Args:
            v1: The first vertex.
            v2: The second vertex.

        Note:
            If either vertex does not exist, it will be added automatically.
            This implementation assumes an undirected graph.
```

Output:

```
arepallyramcharan@Arepallys-MacBook-Air assignment-11.5.py % /usr/bin/python3 "/Users/arepallyr
amcharan/ai assistent.py/assignment-11.5.py/task5.py"
Adjacency List:
A: ['B', 'C']
B: ['A', 'D']
C: ['A', 'D']
D: ['B', 'C']
arepallyramcharan@Arepallys-MacBook-Air assignment-11.5.py %
```

Observation:

1. Graphs represent relationships between nodes using edges.
2. Adjacency lists efficiently store graph connections.
3. Methods like add_vertex and add_edge help construct the graph.
4. Graphs are widely used for network and path representation.
5. AI assistance helps generate structured graph implementations.

Task Description #6: Smart Hospital Management System – Data Structure Selection

A hospital wants to develop a Smart Hospital Management System that handles:

1. Patient Check-In System – Patients are registered and treated in order of arrival.

2. Emergency Case Handling – Critical patients must be treated first.

3. Medical Records Storage – Fast retrieval of patient details using ID.

4. Doctor Appointment Scheduling – Appointments sorted by time.

5. Hospital Room Navigation – Represent connections between wards and rooms.

Student Task

• For each feature, select the most appropriate data structure from the list below:

o Stack

o Queue

o Priority Queue

o Linked List

o Binary Search Tree (BST)

o Graph

o Hash Table

o Deque

• Justify your choice in 2–3 sentences per feature.

• Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.

• A functional Python program implementing the chosen feature with comments and docstrings.

task6.py > ...

```python

import heapq

class EmergencyCaseHandler:
    def __init__(self):
        """Initialize an empty priority queue for emergency cases."""
        self._queue = []
        self._counter = 0  # To break ties for same severity

    def add_case(self, patient_name, severity):
        """
        Add a new emergency case.
        Args:
            patient_name (str): Name of the patient.
            severity (int): Severity level (higher = more urgent).
        """
        # Use negative severity for max-heap behavior
        heapq.heappush(self._queue, (-severity, self._counter, patient_name))
        self._counter += 1

    def process_next_case(self):
        """
        Process the next most severe case.
        Returns:
            str: Name of the patient processed, or None if queue is empty.
        """
        if not self._queue:
            print("No emergency cases to process.")
            return None
        severity, _, patient_name = heapq.heappop(self._queue)
        print(f"Processing emergency case: {patient_name} (Severity: {-severity})")
        return patient_name

# Example usage
if __name__ == "__main__":
```

Output:

```
arepallyramcharan@Arepallys-MacBook-Air assignment-11.5.py % /usr/bin/python3 "/Users/arepallyr
amcharan/ai assistent.py/assignment-11.5.py/task6.py"
Processing emergency case: Patient B (Severity: 8)
Processing emergency case: Patient A (Severity: 5)
Processing emergency case: Patient C (Severity: 3)
No emergency cases to process.
arepallyramcharan@Arepallys-MacBook-Air assignment-11.5.py %
```

Observation :

1. Different hospital operations require different data structures.
2. Queue is suitable for handling patient check-in processes.
3. Priority queues help manage emergency cases efficiently.
4. Hash tables allow quick retrieval of patient medical records.
5. Graph structures help represent connections between hospital rooms.

Task Description #7: Smart City Traffic Control System

A city plans a Smart Traffic Management System that includes:

1. Traffic Signal Queue – Vehicles waiting at signals.

2. Emergency Vehicle Priority Handling – Ambulances and fire trucks prioritized.

3. Vehicle Registration Lookup – Instant access to vehicle details.

4. Road Network Mapping – Roads and intersections connected logically.

5. Parking Slot Availability – Track available and occupied slots.

Student Task

• For each feature, select the most appropriate data structure from the list below:

o Stack

o Queue

o Priority Queue

o Linked List

o Binary Search Tree (BST)

o Graph

o Hash Table

o Deque

• Justify your choice in 2–3 sentences per feature.

• Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.

• A functional Python program implementing the chosen feature with comments and docstrings.

task7.py > ...

```python

from queue import PriorityQueue

class Vehicle:
    """
    Represents a vehicle in the traffic system.
    Attributes:
        reg_number (str): Vehicle registration number.
        is_emergency (bool): True if vehicle is emergency (ambulance, fire, police).
        priority (int): Lower value means higher priority.
    """
    def __init__(self, reg
                       (parameter) reg_number: Any
        self.reg_number = reg_number
        self.is_emergency = is_emergency
        # Emergency vehicles get higher priority (lower number)
        self.priority = 0 if is_emergency else 1

    def __lt__(self, other):
        # PriorityQueue uses this for ordering
        return self.priority < other.priority

    def __repr__(self):
        return f"{'[EMERGENCY]' if self.is_emergency else '[NORMAL]'} {self.reg_number}"

class EmergencyVehiclePriorityQueue:
    """
    Priority queue for vehicles, prioritizing emergency vehicles.
    """
    def __init__(self):
        self.queue = PriorityQueue()

    def add_vehicle(self, vehicle):
        """
        Add a vehicle to the queue.
        """
```

Output:

```
arepallyramcharan@Arepallys-MacBook-Air assignment-11.5.py % /usr/bin/python3 "/Users/arepallyr
amcharan/ai assistent.py/assignment-11.5.py/task7.py"
Processing vehicles in priority order:
[EMERGENCY] AMB001
[EMERGENCY] FIRE002
[NORMAL] ABC123
[NORMAL] XYZ789
arepallyramcharan@Arepallys-MacBook-Air assignment-11.5.py %
```

Observation:

- Queue structures efficiently manage vehicles waiting at traffic signals.
- Priority queues help handle emergency vehicles first.
- Hash tables enable quick lookup of vehicle registration data.

- Graphs represent road networks and intersections effectively.
- Appropriate data structures improve traffic management efficiency.

Task Description #8: Smart E-Commerce Platform – Data Structure Challenge

An e-commerce company wants to build a Smart Online Shopping System with:

1. Shopping Cart Management – Add and remove products dynamically.

2. Order Processing System – Orders processed in the order they are placed.

3. Top-Selling Products Tracker – Products ranked by sales count.

4. Product Search Engine – Fast lookup of products using product ID.

5. Delivery Route Planning – Connect warehouses and delivery locations.

Student Task

• For each feature, select the most appropriate data structure from the list below:

o Stack

o Queue

o Priority Queue

o Linked List

o Binary Search Tree (BST)

o Graph

o Hash Table

o Deque

• Justify your choice in 2–3 sentences per feature.

• Implement one selected feature as a working Python program with

AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.

• A functional Python program implementing the chosen feature

with comments and docstrings.

task8.py > ...

```python

# Implementation: Order Processing System using Queue

from collections import deque

class OrderProcessingSystem:
    """
    A simple order processing system using a queue.
    Orders are processed in the order they are received (FIFO).
    """

    def __init__(self):
        # Initialize an empty queue for orders
        self.order_queue = deque()

    def add_order(self, order_id):
        """
        Add a new order to the processing queue.
        :param order_id: Unique identifier for the order
        """
        self.order_queue.append(order_id)
        print(f"Order {order_id} added to the queue.")

    def process_next_order(self):
        """
        Process the next order in the queue.
        :return: The processed order ID or None if queue is empty
        """
        if self.order_queue:
            order_id = self.order_queue.popleft()
            print(f"Order {order_id} processed.")
            return order_id
        else:
            print("No orders to process.")
            return None
```

Output:

```
arepallyramcharan@Arepallys-MacBook-Air assignment-11.5.py % /usr/bin/python3 "/Users/arepallyr
amcharan/ai assistent.py/assignment-11.5.py/task8.py"
Order ORD001 added to the queue.
Order ORD002 added to the queue.
Order ORD003 added to the queue.
Pending orders: ['ORD001', 'ORD002', 'ORD003']
Order ORD001 processed.
Pending orders: ['ORD002', 'ORD003']
arepallyramcharan@Arepallys-MacBook-Air assignment-11.5.py %
```

Observation:

1. Linked lists help manage dynamic shopping cart operations.
2. Queue structures ensure orders are processed in sequence.
3. Priority queues help track top-selling products efficiently.
4. Hash tables provide fast product search using product IDs.
5. Graph structures help model delivery routes between locations.